

SPECIFICATION

LUKO (ERC-20), Base mainnet · Document version 1.0 · Status: Final

1 · INTRODUCTION 2 · DESIGN PRINCIPLES 3 · TECHNICAL SPECIFICATION
4 · GENESIS 5 · ADMINISTRATIVE MODEL 6 · NON-GOALS 7 · INTENDED AUDIENCE
8 · FAQ 9 · REFERENCES

1 · INTRODUCTION

[↑ TOP](#)

LUKO is a fungible token conforming to the ERC-20 standard, deployed on Base, an Ethereum layer-2 network.

The token was created by a single deployment transaction. That transaction minted the entire supply, assigned it to the deployer address, and terminated the issuance process permanently. No further contract-level action is possible.

LUKO has no administrator, no upgrade mechanism, no treasury logic, no governance module, and no dependency on any off-chain service. The deployed bytecode is the complete and final implementation. There is no second component, no backend, and no protocol layer built on top of it.

This document describes what the contract is and what it does. It does not describe intentions, plans, or expectations, because none exist at the contract level. Readers evaluating LUKO — block explorer reviewers, wallet maintainers, listing desks, auditors — should treat the verified source code as authoritative and this document as a description of it.

LUKO is complete. Nothing is pending.

2 · DESIGN PRINCIPLES

[↑ TOP](#)

The design follows four rules.

Minimality. The contract implements the ERC-20 interface and one public constant, and nothing else. No hooks, no fees, no transfer restrictions, no allowlists, no blocklists, no pause switch, no rebasing, no reflection, no callbacks into external contracts. Every line of behaviour not required by the standard was excluded deliberately.

Immutability as the product. Most token contracts treat immutability as a property to be mentioned. Here it is the entire deliverable. What LUKO offers is the guarantee that its state transition rules cannot be altered by anyone, including its author. That guarantee is only meaningful if there is nothing else — no admin key that could be added, no proxy that could be repointed, no governance that could vote. Removing capability is what creates the guarantee.

Absence of privilege. No address holds a role that other addresses do not. The deployer has no residual rights. After deployment the deployer is an ordinary holder, distinguishable only by balance, not by permission.

Finality over iteration. The contract will not be improved, extended, migrated, or replaced by its author. A version 2 would contradict the premise of version 1. The absence of a roadmap is a design decision, not an omission.

Standard implementation code was used rather than custom code. Novelty in an ERC-20 implementation is a liability, not a feature.

3 · TECHNICAL SPECIFICATION

[↑ TOP](#)

Name	LUKO
Symbol	LUKO
Standard	ERC-20
Network	Base (Ethereum L2)
Chain ID	8453
Contract	0x4a9DA2831A691E7C4aca594CaFd58c35e0131fD1
Decimals	18
Total supply	1,000,000
Total supply (base units)	1,000,000,000,000,000,000,000,000
Supply model	Fixed, minted once at deployment
Implementation	OpenZeppelin ERC20
Language	Solidity, pragma ^0.8.20

License	MIT
Source verification	Verified on BaseScan (exact match)
Mutability	Immutable; not upgradeable
Proxy	None
Owner / admin	None
Mint function	None reachable post-deployment
Burn function	Not exposed
Transfer fee	None
Pause / freeze	None
External calls	None

Implemented interface: `name()`, `symbol()`, `decimals()`, `totalSupply()`, `balanceOf(address)`, `transfer(address,uint256)`, `allowance(address,address)`, `approve(address,uint256)`, `transferFrom(address,address,uint256)`, plus the `Transfer` and `Approval` events.

The contract is not `Ownable`. It contains no `onlyOwner` modifier, no role registry, no `AccessControl` inheritance, and no `selfdestruct`. It holds no ETH-handling logic and has no `receive` or `payable` function.

Total supply is set once, in the constructor, and is thereafter a constant of the deployed state. `totalSupply()` will return the same value for the lifetime of the network.

The only member beyond the standard interface is `INITIAL_SUPPLY`, a public `uint256` constant equal to the genesis supply in base units. It is compile-time constant and read-only.

Behaviour matches the OpenZeppelin reference implementation exactly; no function was overridden. Integrators may therefore rely on standard ERC-20 assumptions without special handling. Note the standard `approve` race condition, inherent to ERC-20 itself and not specific to this deployment.

3.1 · SOURCE

The verified source is reproduced below in full. It is the entire contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
contract LUK0 is ERC20 {
    uint256 public constant INITIAL_SUPPLY = 1_000_000 ether;

    constructor() ERC20("LUK0", "LUK0") {
        _mint(msg.sender, INITIAL_SUPPLY);
    }
}
```

The ether suffix is a Solidity literal unit denoting 10^{18} and carries no monetary meaning; it expresses 1,000,000 tokens at 18 decimals.

4 · GENESIS

[↑ TOP](#)

The entire supply of 1,000,000 LUKO was minted in the contract's constructor during the deployment transaction, included in Base block **49364826**.

That mint is the only issuance event that has occurred and the only one that can occur. The minting code path exists solely inside the constructor and is unreachable once the contract is deployed. There is no scheduled emission, no vesting built into the contract, no inflation parameter, no staking rewards, and no mechanism by which `totalSupply()` can increase.

All minted tokens were credited to the deploying address at genesis. Subsequent distribution, if any, has occurred through ordinary transfers recorded on-chain. Current holder distribution is observable on any Base block explorer and is not restated here, as it changes independently of this document.

Tokens sent to unrecoverable addresses are permanently removed from circulation. The contract exposes no burn function; supply reduction is possible only in that incidental sense.

Some tokens transferred after genesis have been placed into non-cancelable Sablier vesting streams. Stream terms, once created, are permanent and independently verifiable on-chain; the specification does not enumerate individual streams, as recipients and schedules are distribution detail outside its scope.

5 · ADMINISTRATIVE MODEL

[↑ TOP](#)

There is no administrative model. This section exists to state that explicitly, because its absence is otherwise indistinguishable from an omission.

The contract has no owner. Ownership functionality was not renounced after deployment — it was never included. This distinction matters: a renounced-ownership contract passed through a window in which privileged action was possible. LUKO had no such window.

Consequently:

- No address can mint tokens.
- No address can burn or seize the tokens of another address.
- No address can pause, freeze, or restrict transfers.
- No address can modify balances, allowances, metadata, or supply.
- No address can upgrade, replace, or migrate the contract.
- No address can extract funds from the contract, which holds none.

There is no proxy, no implementation slot, no beacon, and no `delegatecall`. The address a user interacts with contains the executing logic directly. The code returned by `eth_getCode` at that address is the code that will run, permanently.

The author retains no capability over the contract beyond that of any other address. There is no key whose compromise could alter the token's behaviour, because no such key exists.

6 · NON-GOALS

[↑ TOP](#)

LUKO is explicitly **not** the following:

- **Not a governance token.** It confers no voting rights over anything, on-chain or off-chain. There is no DAO, no proposal system, no snapshot process.
- **Not equity.** It represents no ownership of, claim on, or entitlement to any entity, asset, revenue, dividend, or distribution.
- **Not a protocol.** There is no lending, exchange, staking, bridging, or settlement layer. A single ERC-20 contract is the whole system.
- **Not an investment vehicle.** No return is offered, implied, projected, or managed. There is no fund, no strategy, and no party acting on a holder's behalf.
- **Not a promise of future utility.** No feature is planned. No integration is committed. No product will be built to give the token a use.
- **Not a claim to solve a problem.** LUKO does not assert a market inefficiency, a technical gap, or a use case.
- **Not a security-audited guarantee of user outcomes.** The implementation is standard and the source is verified; neither fact protects a holder from their own transactions, from price, or from third-party contracts they choose to interact with.

The absence of utility is intentional and permanent. It should be read as a specification, not as an early stage.

LUKO is not designed to satisfy every possible expectation of an ERC-20 token.

The intended audience is not defined by what they expect to be added, but by what they accept will never be added. For those who consider the deliberate absence of governance, utility, administrative control and future obligations to be features rather than deficiencies.

Why 1,000,000?

Because a fixed number was required and a small round one is legible. One million units at 18 decimals is unambiguous, easy to reason about, and requires no explanation of emission curves. No modelling justifies the figure; none was performed. The specific number matters less than the fact that it cannot change.

Why no owner?

An owner is a trust assumption. Any privileged address must be trusted not to use its privileges, and trusted to keep its key secure indefinitely. Omitting the role removes both risks at the cost of removing all administrative flexibility — flexibility this contract does not need.

Why immutable?

Upgradeability means the rules a holder agreed to can be replaced later by someone else. Immutability means the rules verified today are the rules that apply permanently. For a contract with no functionality beyond balance accounting, there is nothing to upgrade toward, so upgradeability would add only risk.

Why is the initial liquidity pool so small?

The liquidity pool exists to establish a public market and enable permissionless transfers between holders. It is not intended to provide deep liquidity, support speculation, or facilitate large-scale trading. LUKO is not structured as an investment product, and the pool should not be interpreted as one.

Why are so many tokens held in reserve?

Tokens held in reserve are simply held. They confer no additional rights, generate no yield, and possess no administrative capability. A token in reserve is technically indistinguishable

from a token in any other wallet. Reserve balances exist only as balances, not as protocol-controlled assets.

Why disclose the genesis allocation?

Because the initial distribution is part of the artifact. Publishing it allows every subsequent transfer to be interpreted in the context of the original state.

Why no roadmap?

A roadmap is a set of unenforceable commitments about future work. Publishing one would create expectations the contract cannot guarantee, and the contract is the only thing that can be guaranteed. The deployment is the deliverable, and it is finished.

Why no utility?

Utility implies a system the token operates within, and that system would require maintenance, operators, and continued trust — the properties this design removes. LUKO is defined by what it does not depend on. Attaching a use case would reintroduce the dependencies.

Why Base?

Base is an Ethereum layer-2 with EVM equivalence, low transaction costs, standard tooling, and established explorer and wallet support. It requires no custom client, no non-standard token behaviour, and no bespoke integration work. No property of LUKO depends on Base specifically; it is a deployment target, not a component of the design.

9 · REFERENCES

[↑ TOP](#)

PRIMARY

Contract address [0x4a9DA2831A691E7C4aca594CaFd58c35e0131fD1](#)

Block explorer (BaseScan) basescan.org/token/0x4a9D...1fD1 →

Holder distribution (BaseScan) basescan.org/token/tokenholderchart/0x4a9D...1fD1 →

Source repository (GitHub) github.com/LKSPEC/luko →

X x.com/MeetLUKO →

Website meetluko.eu →

Genesis block 49364826

STANDARDS AND NETWORK

Standard ERC-20 (EIP-20)

Implementation reference [OpenZeppelin Contracts — ERC20](#)

Network base.org →

THIRD-PARTY VENUES AND DATA SOURCES

The following are operated by unaffiliated parties. They are listed because they reference the contract address, not because they are endorsed, controlled, or maintained by the token's author. Their availability, pricing, and liquidity are outside the scope of this specification.

Aerodrome (DEX) aerodrome.finance →

DexScreener dexscreener.com →

GeckoTerminal geckoterminal.com →

In case of any discrepancy between this document and the verified source code at the contract address, the source code is authoritative.

BUILT BY LKSPEC · ALL IS STATED · NOTHING IS MISSED · LOOK AGAIN

LKSPec is the publishing identity of [Denisas Bykovas](#). · x.com/MeetLUKO